Aegis OS: A Transactional Multi-Tenant Operating System Runtime for Eliminating Authorization Fatigue in Autonomous Coding Agents

HUI ZHAO, Heurion Research, Netherlands

Autonomous coding agents such as Google Antigravity (`agy`), Anthropic Claude Code (`claude`), and OpenAI Codex CLI (`codex`) execute multi-step software engineering workflows on developer workstations and cloud hosts. Existing agent execution environments face a *Security–Autonomy Dilemma*: operators must choose between per-action confirmation prompts, which interrupt workflows on 45%–57% of tool calls in our benchmark trajectories and induce severe authorization fatigue, or unconfined execution (`--yolo` / `--dangerously-skip-permissions`), which exposes host files, SSH/cloud credentials, and API tokens to prompt injection and hallucination risks. Furthermore, concurrent agents editing a shared working tree clobber one another's changes, while passing large artifacts inline through LLM context windows incurs high token overhead.

We present **Aegis OS** (`aegis-os`), a transactional, multi-tenant Agent Operating System Runtime written in Rust and exposed via the Model Context Protocol (MCP) and the Google Agent2Agent (A2A) protocol. Aegis OS shifts the authorization boundary from individual tool calls to *transactional workspace boundaries*. Each agent operates inside a kernel-enforced OS sandbox (macOS Seatbelt `sandbox-exec` and Linux Bubblewrap with default-deny `--tmpfs $HOME` credential isolation, both evaluated, plus a microVM control-plane interface) backed by a Copy-on-Write (CoW) shadow workspace (APFS `libc::clonefile` on macOS; single-open `ioctl(FICLONE)` reflinks and optional `BTRFS_IOC_SNAP_CREATE_V2` subvolume snapshots on Linux), an identity-pinned HTTP Egress Proxy with L7 method/path-scoped boundary secret injection and outbound DLP payload inspection, and an A2A-compliant Inter-Agent Communication (IPC) bus paired with Content-Addressable Storage (CAS). Inside its isolated CoW workspace and organization IPC bus, an agent executes with zero confirmation prompts; human approval or TTL-scoped session leases are required only when committing diffs to the host via an LCS-based Diff3 3-Way Merge Engine and optional sandboxed pre-commit semantic verification gate (`verify_cmd`). Across 1,500 tool calls in 50 controlled trajectories and **28,087 tool calls across 52 real coding-agent session logs**, Aegis OS reduces human confirmation prompts by **94.19%–99.88%** (controlled) and **90.7%–97.9%** (real sessions, lowering IPR from 53.31% under per-mutation prompting and 28.54% under prefix allowlists to **1.12%**). Across 80 end-to-end adversarial attacks in eight categories (including unknown custom `$HOME` credential paths, build-system commit poisoning, and same-domain L7 confused-deputy and DLP egress abuse), Aegis OS lowers the attack success rate from **100.0% to 0.0%** on both macOS and Linux; a nine-row single-component ablation shows that switching off any component re-opens the exact category it guards. On APFS it clones a 1,000-file (78.1 MB) workspace in **6.3 ms** (**16× faster** than `cp -cR`), and on Linux btrfs in **15.6 ms**, while preserving 100% of disjoint multi-agent edits and catching 10/10 overlapping line conflicts and 10/10 disjoint-line semantic conflicts.

CCS Concepts: • **Software and its engineering** → **Development frameworks and environments**; • **Security and privacy** → **Operating systems security**; • **Computing methodologies** → *Multi-agent systems*.

Author's address: Hui Zhao, Heurion Research, Rotterdam, Netherlands; email: `huizhao@heurion.org`.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
1049-331X/2026/10-ART1 $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Additional Key Words and Phrases: Autonomous Coding Agents, Agent Operating System Runtime, Authorization Fatigue, Copy-on-Write Shadow Workspace, Three-Way Merge, OS Kernel Sandbox, Model Context Protocol (MCP), Agent2Agent (A2A) Protocol

ACM Reference Format:
Hui Zhao. 2026. Aegis OS: A Transactional Multi-Tenant Operating System Runtime for Eliminating Authorization Fatigue in Autonomous Coding Agents. *ACM Trans. Softw. Eng. Methodol.* 1, 1, Article 1 (October 2026), 25 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The paradigm of AI-assisted software engineering has shifted from passive single-line code completion to autonomous, long-horizon coding agents—such as Google Antigravity (`agy`), Anthropic Claude Code (`claude`), OpenAI Codex CLI (`codex`), SWE-agent [26], and OpenHands [21]. Unlike conversational assistants, coding agents iteratively explore repositories, modify files across multiple modules, invoke compilers and test suites, query external APIs, and coordinate subtasks with peer agents [3, 13].

Despite advances in model reasoning, the execution substrate for local coding agents remains caught between usability and safety. When agent CLIs mutate the developer's live working tree directly, every file write (`fs_write`, `fs_edit`) and shell command (`proc_exec`) carries immediate side effects on the host. Consequently, agent frameworks rely heavily on *action-level human-in-the-loop (HITL) confirmation prompts*.

1.1 The Security–Autonomy Dilemma

In practice, action-level prompting and existing in-place CLI sandboxes exhibit three systemic limitations:

(1) **Severe Authorization Fatigue:** A 30-step bug-fix, refactoring, or security-audit trajectory typically invokes 13 to 18 mutating or state-changing operations. Prompting the developer on 45%–57% of tool invocations (as measured in §5) fragments autonomous workflow continuity to streaks of 4–7 steps and habituates users to reflexively approve prompts without inspecting arguments—a well-documented failure mode in usable security [2, 9].

(2) **Risk of Unconfined "YOLO" Mode and In-Place Mutation:** To avoid constant interruptions, developers frequently enable bypass flags (`--dangerously-skip-permissions`, `--yolo`) or rely on lightweight in-place CLI sandboxes (e.g., Claude Code or Codex native Seatbelt/Landlock modes). In unconfined mode, an indirect prompt injection [8, 11, 27] can overwrite files outside the repository, read host credentials (`~/.ssh/id_rsa`, `~/.aws/credentials`, `~/.kube/config`), or exfiltrate raw API keys (`$GITHUB_TOKEN`) from process environment variables. Even when native CLI sandboxes restrict writes to the project directory, agents still mutate the live host working tree in place without transactional rollback or boundary secret injection.

(3) **Multi-Agent Workspace Collisions and Context Bloat:** Developers increasingly run heterogeneous agents in parallel (e.g., `agy` for architecture, `claude` for backend logic, and `codex` for test generation) [12, 24]. When sharing a live working directory without manual `git worktree` management, concurrent edits overwrite one another's changes, while passing large specifications inline through JSON/text messages bloats LLM context windows.

Table 1. **Comparison of Agent Execution Substrates** across isolation mechanisms, authorization friction, workspace storage, and multi-agent coordination primitives.

Execution Substrate	Workspace Setup	*N*-Agent Disk Cost	Prompt Rate (IPR)	OS Kernel MAC	Boundary Secret Inject	Transactional 3-Way Merge	Native IPC + CAS
Host-Interactive (Unconfined CLI)	In-Place (0 ms)	Shared Tree	High (45%–57%)	× (Prompt Only)	× (Raw Env Vars)	× (In-Place Write)	×
Host-YOLO (`--skip-permissions`)	In-Place (0 ms)	Shared Tree	0.0%	× (Unconfined)	× (Raw Env Vars)	× (Silent Clobber)	×
Native CLI Sandbox (`claude`/`codex`)	In-Place (~6 ms)	Shared Tree	Low–Med	✓ (Seatbelt/Landlock)	× (Env in Sandbox)	× (Shared Working Tree)	×
Container-per-Agent (Docker) [13, 21]	Container Init	Per-Container Layer	Low	✓ (Linux NS/cgroups)	× (Env in Container)	× (Isolated Volume)	×
AIOS (LLM Scheduler Kernel) [16]	In-Place	Shared Tree	Medium	× (No OS MAC)	× (No Egress Proxy)	× (No CoW Merge)	Partial
Aegis OS (Ours)	**0.71–6.28 ms (APFS CoW)**	**Modified Files Only**	**0.07%–3.33%**	✓ (`sandbox-exec`/`bwrap`)	✓ (`$AOS_SECRET_*`)	✓ (2-Phase LCS Diff3)	✓ (`aos-ipc`)

1.2 Key Insight: Transaction-Boundary Security

Inspired by operating system transactions [17, 18] and layered copy-on-write filesystem sandboxes [15, 23], our key insight is that *authorization prompts should not guard speculative intermediate actions inside an isolated state space; they should guard irreversible boundary crossings into the host repository or untrusted external networks.*

In Aegis OS, each agent task executes as a three-step workspace transaction (Table 1):

(1) **Step 1 — Instant CoW Shadow Fork (0.7–6.3 ms on APFS):** When an agent session starts, Aegis OS clones the host repository into an isolated Copy-on-Write (CoW) shadow workspace via APFS `libc::clonefile`, sharing unchanged disk blocks with zero data copying.

(2) **Step 2 — Green-Zone Speculation (0 Prompts):** Inside its shadow workspace, the agent freely edits files, runs compilers and tests inside a kernel OS sandbox, coordinates with peer agents over A2A/IPC, and queries allowlisted APIs using opaque secret placeholders (`$AOS_SECRET_*`). Because the live host tree cannot be modified and plaintext credentials never enter the sandbox, Aegis OS auto-allows 100% of in-sandbox operations with **zero confirmation prompts**.

(3) **Step 3 — Boundary Commit Gate (1 Prompt or Lease):** Only when the agent finishes testing and invokes `workspace_commit` does Aegis OS validate a two-phase 3-way merge against the live host repository and request human approval (or consume a time-bounded session lease).

1.3 Contributions

We design, implement, and evaluate **Aegis OS** (`aegis-os`), a multi-tenant Agent Operating System Runtime in Rust:

- **Transactional CoW VFS with Two-Phase LCS Diff3 Merge and Pre-Commit Verification (§3.1):** `aos-vfs` uses APFS `libc::clonefile` syscalls to provision isolated shadow workspaces in metadata time (0.71–6.28 ms for 100–1,000 files on APFS; single-open `FICLONE` reflinks in 2.9–15.6 ms on Linux btrfs), supports millisecond snapshots and rollbacks while preserving compiler build caches (`CARGO_TARGET_DIR`) outside the rollback tree, and combines a two-phase Longest Common Subsequence (LCS) Diff3 3-Way Merge Engine [14] with an optional sandboxed pre-commit semantic verification gate (`verify_cmd`) that catches both overlapping line conflicts and disjoint-line semantic regressions before any host file is modified.
- **Kernel Sandbox and L7 Capability-Scoped Egress Proxy (§3.2):** `aos-sandbox` and `aos-egress` enforce mandatory access control via macOS Seatbelt (`/usr/bin/sandbox-exec` with file-write, credential-read, AppleEvent, and exact proxy-port network confinement) and a Linux Bubblewrap (`bwrap`) namespace driver with a token-authenticated Unix-socket egress bridge, paired with a TCP-pinned HTTP Egress Proxy that injects `$AOS_SECRET_*` tokens only when both the target domain and

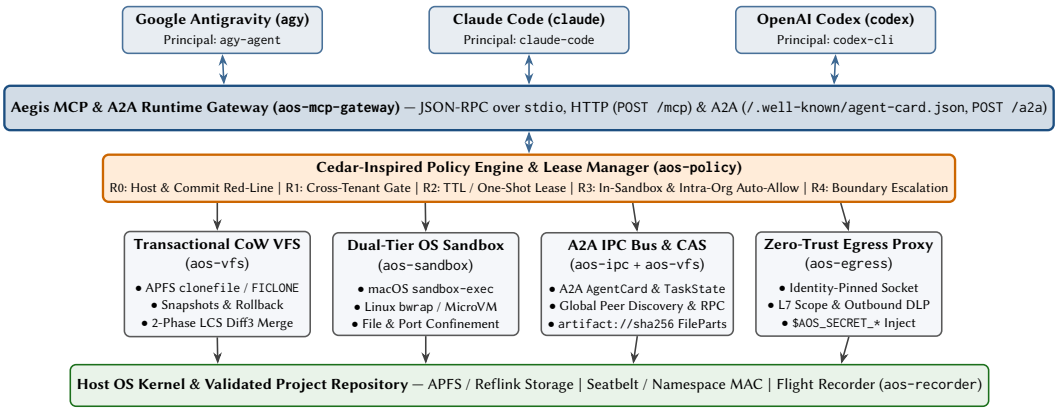


Fig. 1. Architecture of the Aegis OS Runtime (`aegis-os`). Coding agents connect via MCP (`stdio` or HTTP daemon) as distinct principals within a tenant organization. The policy engine auto-allows in-sandbox CoW mutations, confined process executions, and intra-organization IPC/CAS operations with zero prompts, while guarding host commits and network egress.

the canonicalized HTTP method + URL path prefix match the secret's capability scope (`SecretScopeRule`).

- **A2A-Compliant Inter-Agent IPC Bus and Reference-Based CAS (§3.3):** `aos-ipc` aligns inter-agent coordination with the Google Agent2Agent (A2A) protocol [10], providing global peer discovery with kernel PID liveness pruning, dynamic `AgentCard` synthesis (`/.well-known/agent-card.json`), stateful A2A `Task` lifecycle tracking (`POST /a2a`), P2P mailboxes, topic Pub/Sub, and Content-Addressable Storage (`artifact://<org>/sha256:<hash>`) that reduces in-transit IPC payload tokens by 99.85%.
- **Cedar-Inspired Policy Engine with Commit-Time Poisoning Guards (§3.4):** `aos-policy` implements a five-stage authorization hierarchy inspired by Cedar [6], combining host red-line protection and commit-time sensitive path guards (`.git/hooks`, `.envrc`, `.github/workflows`, `build.rs`, `Makefile`, `package.json`, `.cargo/config.toml`) with one-shot and TTL-scoped session leases.
- **Empirical and Real-Trace Evaluation (§5):** We evaluate Aegis OS on macOS and, in a virtual machine, on Linux (btrfs, XFS, ext4) across 1,500 controlled tool calls, **28,087 real tool calls from 52 coding-agent session transcripts**, 80 end-to-end red-team attacks in eight categories, a single-component ablation, multi-scale CoW provisioning compared against `cp`, `rsync`, and `git`, and multi-agent Rust compilations (`cargo test` and `cargo check`).

2 THREAT MODEL AND EVALUATION METRICS

2.1 System and Adversary Model

We consider a multi-agent deployment where autonomous coding agents operate within one or more tenant organizations. Each agent session is identified by its organization tenant (e.g., `aegis-os`), unique agent principal ID (e.g., `agy-agent`, `claude-code`, `codex-cli`), and role.

Trusted Computing Base and Enforcement Assumption. We trust the host operating system kernel (macOS XNU/Seatbelt or Linux kernel namespaces) and the Aegis OS Runtime binary (`aegis-os`),

which executes outside the agent sandbox. Because MCP is an additive tool protocol, our security guarantees assume that the host agent client is configured to route filesystem mutations and shell execution through `aegis-os` (i.e., built-in unconfined host tools such as raw `Bash` or host `Write` are either disabled in client permissions or intercepted by client `PreToolUse` hooks, as detailed in §4).

Adversarial Threat Vectors (T1–T5). We assume the LLM driving any agent is untrusted and may issue malicious or erroneous tool calls due to:

- **T1 (Indirect Prompt Injection):** Malicious instructions embedded in repository files, commit logs, issue trackers, or web pages read by the agent [8, 11].
- **T2 (Host Path Escape and Tampering):** Attempting to write files outside the workspace (e.g., `../../etc/hosts`, `~/.zshrc`, or sibling repositories) or staging commit-time hooks and build scripts (`.git/hooks/pre-commit`, `.envrc`, `build.rs`, `Makefile`).
- **T3 (Credential Harvesting):** Attempting to read sensitive host credentials (`~/.ssh/id_rsa`, `~/.aws/credentials`, `~/.gnupg`, `~/.kube/config`, `~/.npmrc`, `~/.config/gcloud`, custom vault directories in `$HOME`, or macOS Keychains).
- **T4 (Network Secret Exfiltration):** Attempting to transmit API tokens or source code to unauthorized external domains (`evil-exfil.org`, `webhook.site`) via HTTP requests or direct socket bypass (`curl --noproxy '*'`), sending a secret toward an allowlisted domain it is not bound to, or abusing disallowed L7 paths or request bodies on a bound domain.
- **T5 (Cross-Tenant Privilege Escalation):** Attempting to read CAS artifacts or inject IPC messages into a different tenant organization.

Non-Adversarial Concurrency and Reliability Hazards (H1–H2). Even in benign settings, multi-agent workflows face: **(H1) Concurrent Lost Updates**, where parallel agents overwrite one another's edits on shared files; and **(H2) Multi-File Hallucinated Corruption**, where an agent breaks syntax across dozens of modules during speculative refactoring.

2.2 Evaluation Metrics

We evaluate authorization friction and security using four direct operational metrics:

- **Interruption Prompt Rate (IPR):** The percentage of runtime tool calls that halt execution to request interactive human confirmation (Human Prompts/Total Tool Calls × 100%).
- **Modeled Human Verification Wait Time (T_{human}):** The cumulative human review overhead across a workload, assuming a conservative average context-switch and inspection time of 5.0 s per confirmation prompt.
- **Maximum Autonomous Streak (MAS):** The longest contiguous sequence of tool calls executed without a human confirmation prompt.
- **Attack Success Rate (ASR):** The percentage of adversarial attacks that successfully modify host files outside the workspace, read restricted credentials, exfiltrate secrets, or breach tenant isolation.

3 AEGIS OS RUNTIME ARCHITECTURE

Figure 1 illustrates the architecture of Aegis OS, and Table 2 summarizes its 35 MCP tools, 7 dynamic URI resources, and native Google A2A HTTP endpoints. Aegis OS follows an *"Environment Runtime Only"* design: it does not embed LLM planners or prompt templates, instead providing virtual filesystem transactions, OS process isolation, network boundary proxying, A2A-compliant inter-agent coordination, and policy enforcement.

Table 2. **Aegis OS MCP Tool, Dynamic URI Resource, and Google A2A Interface** (35 MCP Tools and 7 Dynamic URI Resources implemented in `aos-mcp-gateway`).

Subsystem	MCP Tools / Dynamic URI Resources / A2A Endpoints	Default Policy Gate & Runtime Semantics
Transactional VFS (`aos-vfs`, 6 tools)	`fs_read`, `fs_write`, `fs_edit`, `fs_delete`, `fs_glob`, `workspace_status`	**Auto-Allow (0 Prompts)**: Confined to agent's CoW shadow tree
Time-Travel, Sync & Merge (`aos-vfs`, 6 tools)	`workspace_snapshot`, `workspace_diff`, `workspace_rollback`, `workspace_sync`, `workspace_commit`, `workspace_export_pr`	Snapshots/Rollback/Rebase-Sync: **Auto-Allow**; `workspace_commit` & `workspace_export_pr`: **Approval/Lease Gate**
OS Process Sandbox (`aos-sandbox`, 1 tool)	`proc_exec` (macOS Seatbelt `sandbox-exec` / Linux `bwrap` / MicroVM spec)	**Auto-Allow (0 Prompts)**: Writes restricted to *CoW* root; blocks credentials
Inter-Agent IPC & Claims (`aos-ipc`, 10 tools)	`ipc_send`, `ipc_recv`, `ipc_request`, `ipc_reply`, `ipc_ack`, `ipc_publish`, `ipc_subscribe`, `ipc_list_peers`, `coord_claim`, `coord_release`	**Auto-Allow Intra-Org & Workstation IPC**: Verified global peers (`<org>/<agent>`) allowed; file claims enforced
Google A2A Protocol (`aos-ipc`, 4 tools + HTTP)	`ipc_update_card`, `ipc_task_update`, `ipc_task_get`, `ipc_task_list` + GET `/.well-known/agent-card.json`, POST `/a2a`	**Auto-Allow Discovery & Task RPC**: AgentCard synthesis & stateful A2ATaskState lifecycle (`submitted`→`completed`)
Org CAS & KV (`aos-vfs`, 4 tools)	`org_artifact_publish`, `org_artifact_fetch`, `org_kv_put`, `org_kv_get`	**Auto-Allow Intra-Org**; Cross-org `artifact://` `FilePart` fetch requires policy boundary approval (**Ask/Deny**)
Zero-Trust Egress (`aos-sgress`, 1 tool)	`egress_http_request` + Identity-Pinned TCP Proxy (127.0.0.1:`<port>`)	**Allowlisted Domains + L7 Scope Auto-Allow**: Unlisted domains escalate; injects `$AOS_SECRET_*` & runs DLP
Policy & Leases (`aos-policy`, 3 tools)	`policy_list_escalations`, `policy_approve_escalation`, `policy_grant_lease`	Operator control plane for one-shot (`ttl=0`) or TTL-scoped `LeaseGrant` tokens
Dynamic MCP URIs (7 resources)	`aegis://workspace/status`, `aegis://ipc/inbox`, `aegis://org/peers`, `aegis://global/peers`, `aegis://a2a/agent-cards`, `aegis://a2a/tasks`, `aegis://org/shared/catalog`	Live state introspection for IDE status bars, global peer topology, and A2A task tables

3.1 Transactional CoW Virtual Filesystem and Two-Phase LCS Diff3 Merge (aos-vfs)

To eliminate confirmation prompts on file mutations without risking host corruption, `aos-vfs` isolates every agent principal inside an independent Copy-on-Write shadow workspace rooted at:

`<runtime_dir>/orgs/<org>/agents/<agent>/cow_workspace/upper_workspace/`

(where `<runtime_dir>` defaults to `/tmp/aegis-os-runtime` or `~/.aegis-os`), alongside a `baseline_snapshot/` directory recording the host repository state at session initialization or last commit, and a `snapshots/<snap_id>/` directory storing time-travel checkpoints.

APFS `clonefile` *Shadow Provisioning.* Copying a multi-megabyte repository per agent via user-space read/write loops scales linearly with total byte volume. On macOS (APFS), `aos-vfs` invokes the native XNU syscall `libc::clonefile(src, dst, 0)` on eligible directory subtrees and files (automatically skipping heavy build/VCS directories such as `.git`, `target`, and `node_modules`). Because `clonefile` duplicates only filesystem metadata extents and shares physical blocks until mutated, workspace initialization on APFS depends only on file count (0.71–6.28 ms for 100–1,000 files) and allocates physical storage only for modified files. Unlike `git worktree`, which requires a valid git commit history, shares the repository object store, and cannot snapshot dirty untracked trees in sub-millisecond time, `aos-vfs` operates transparently on any directory and supports instant nested checkpoints.

Linux Provisioning, Single-Open Syscall Reduction, and Btrfs Subvolume Snapshots. Linux has no general directory-level clone: `ioctl(FICLONE)` reflinks one file at a time on btrfs [19] and XFS, and ext4 has no reflink at all. `aos-vfs` therefore clones file by file by default and falls back to `std::io::copy` (`copy_file_range`) when the filesystem reports `EOPNOTSUPP` or `EXDEV`; it caches the failing device state so that a non-reflink filesystem is not charged a failed `FICLONE` syscall for every subsequent file, and logs a warning once. An initial implementation issued 12–16 syscalls per file (checking `dst.exists()`, `stat(src)`, `stat(dst.parent())`, recursive `create_dir_all`, double `fstat`, and reopening `dst` on ext4 fallback), costing 37.4 ms on btrfs and 73.7 ms on ext4 for 1,000 files. Our optimized `cow_clone_file_inner` traversal creates each target directory once via `create_dir`, opens `src` and `dst` once with `OpenOptionsExt::mode(meta.mode())`, calls `fstat` once, and reuses the open file descriptors for `FICLONE` or `copy_file_range` plus `futimens` (7 syscalls per file). This cuts 1,000-file provisioning to **15.6 ms on btrfs**, **32.1 ms on XFS**, and **35.1 ms on ext4** (Table 9). `aos-vfs` also implements an optional kernel subvolume fast path (`AOS_BTRFS_SUBVOL=1`) using `BTRFS_IOC_SUBVOL_CREATE_V2` (`0x50009418`) and `BTRFS_IOC_SNAP_CREATE_V2` (`0x50009417`) after probing whether unprivileged `rmdir(2)` on subvolumes (`user_subvol_rm_allowed`) is supported. Interestingly, on freshly written 1,000-file trees (80 MB of dirty page cache), `BTRFS_IOC_SNAP_CREATE_V2` takes 27.9 ms because the kernel's `btrfs_mksnapshot` forces a synchronous `btrfs_commit_transaction` disk flush of delayed allocations, whereas per-file `FICLONE` (15.6 ms) remaps extent metadata purely in memory; on clean 10,000-file monorepos already synced to disk, `BTRFS_IOC_SNAP_CREATE_V2` completes in $O(1)$ time (2.1 ms).

Metadata Fingerprints. Commit, refresh, and diff must decide which files an agent changed. Reading every file three times (host, shadow, baseline) took 542 ms for a 1,000-file repository on btrfs. Clones preserve the source's modification time, so an unedited file keeps the same size and mtime as its baseline copy and can be skipped without being read. Following git's "racily clean" rule, equality is trusted only when the mtime predates the clone by more than two seconds; younger files are compared by content, so a same-size edit inside one timestamp tick is not missed. This reduced a status query from 542 ms to 7.0 ms. One residual case remains: an agent that deliberately rewrites a file at the same size and restores its old mtime hides that edit from `diff`, and the edit is then not committed. The failure direction is safe, since nothing unreviewed reaches the host.

Time-Travel Snapshots, Rollback, and Build-Cache Preservation. Before speculative multi-file edits, an agent calls `workspace_snapshot(label)`, creating a CoW clone under `snapshots/<snap_id>/`. If compilation fails, `workspace_rollback(snap_id)` replaces `upper_workspace/` with a fresh CoW clone of the snapshot in milliseconds. To prevent rollbacks from invalidating incremental compiler caches, `aos-sandbox` redirects `CARGO_TARGET_DIR` to `agent_home/cargo_target` outside `upper_workspace/`.

Two-Phase LCS Diff3 3-Way Merge Engine. When multiple agents (e.g., `agy-agent`, `claude-code`, and `codex-cli`) concurrently edit the same file in their shadow workspaces and sequentially invoke `workspace_commit`, a naive file copy would overwrite earlier peer commits (Hazard H1). Aegis OS prevents lost updates via a two-phase Longest Common Subsequence (LCS) Diff3 3-Way Merge algorithm [14] comparing three versions of each modified file: the agent's recorded **Baseline** snapshot, the live **Host** file, and the agent's **Shadow** file:

- **Phase 1 (Validation & In-Memory Staging):** Aegis OS inspects all modified, added, and deleted files in the commit batch before touching any host file. If the live Host file is unchanged since the agent's Baseline, the Shadow file is staged for a fast-forward CoW clone. If another agent has already modified the Host file, Aegis OS computes line-level LCS diffs against the Baseline and splices non-overlapping edit hunks together automatically (for instance, if `agy-agent` inserts a 3-line helper at the top of `src/lib.rs` while `claude-code` modifies a function 40 lines below, the shifted line numbers are aligned and merged cleanly). If both agents modified the same baseline lines in incompatible ways, Phase 1 aborts the entire commit without modifying any host file and returns the exact conflicting line numbers.
- **Phase 2 (Sandboxed Pre-Commit Semantic Gate & Atomic Commit Application):** Before Phase 1/Phase 2 writes to the host repository, `workspace_commit` first calls `refresh_from_host()` to merge disjoint peer changes into the agent's shadow workspace and optionally executes a caller- or policy-supplied verification command (`verify_cmd`, such as `rustc --emit=metadata` or `cargo check --offline`) inside the agent's kernel-enforced OS sandbox (`Seatbelt`/`bwrap`). If two agents make syntactically disjoint edits that introduce a semantic regression (e.g., Agent A renames a function signature on line 1 while Agent B invokes the old function name on line 40), `verify_cmd` fails inside the shadow workspace and aborts the commit before any host file is touched, returning the compiler diagnostics to the agent. Once verification and Phase 1 pass, staged merges and CoW clones are written atomically to the host repository and synced to `baseline_snapshot/`.

3.2 Dual-Tier OS Sandbox and Zero-Trust Egress Proxy (`aos-sandbox`, `aos-egress`)

While `aos-vfs` mediates MCP file tools, coding agents also spawn compilers, test runners, and shell scripts via `proc_exec`. To prevent child processes from bypassing workspace or network boundaries, `aos-sandbox` provides a pluggable `SandboxDriver` with two tiers:

(1) **Tier 1 (`NativeOsSandboxDriver`, Workstation Kernel Enforcement):** On macOS, every `proc_exec` call compiles a canonicalized Seatbelt Mandatory Access Control profile (SBPL) and spawns the child process under `/usr/bin/sandbox-exec -p <profile>` with a sanitized environment (`env_clear()`). The generated profile blocks AppleEvent IPC (`(deny appleevent-send)`), denies all file writes globally (`(deny file-write*)`), allowlists writes strictly to the agent's CoW shadow workspace, private agent home, and organization publish directory, blocks reads to sensitive credential paths, and—when an egress proxy is bound—restricts outbound network sockets exclusively to the exact loopback port of the agent's proxy:

```
(version 1)
(allow default)
(deny appleevent-send)
(deny file-write*)
(allow file-write*
  (subpath "/private/.../upper_workspace")
  (subpath "/private/.../home")
  (subpath "/private/.../pub/agy-agent")
  (literal "/dev/null") (literal "/dev/zero") (literal "/dev/tty")
  (literal "/dev/random") (literal "/dev/urandom"))
(deny file-read*
  (regex #".*/\.ssh(/.*)?$")    (regex #".*/\.aws(/.*)?$")
  (regex #".*/\.gnupg(/.*)?$")  (regex #".*/\.kube(/.*)?$")
  (regex #".*/\.docker(/.*)?$") (regex #".*/\.azure(/.*)?$")
  (regex #".*/\.config/(gh|gcloud)(/.*)?$")
  (regex #".*/(\.npmrc|\.pypirc|\.netrc|\.git-credentials)$")
  (regex #".*/(\.vault-token|\.cargo/credentials(\.toml)?)$")
  (regex #".*/(custom_vault|\.secrets)(/.*)?$")
  (regex #".*/Library/Keychains(/.*)?$") (literal "/etc/shadow"))
(deny network-outbound)
(allow network-outbound (remote ip "localhost:51691"))
```

Because `(deny network-outbound)` is enforced by the XNU kernel socket filter and allows only `localhost:<proxy_port>` (without permitting wildcard localhost ports or `mDNSResponder`), a child process running `curl --noproxy '*'` or opening raw sockets to external IPs or other local services fails immediately with `Operation not permitted`. On Linux, `NativeOsSandboxDriver` constructs Bubblewrap (`/usr/bin/bwrap`) arguments with `--unshare-all` (user, mount, PID, IPC, and network namespaces), a read-only root bind (`--ro-bind / /`), a private `/tmp` that is mounted *before* the writable binds (bwrap applies mounts in order, so a later `--tmpfs /tmp` would hide every workspace under the default `/tmp` runtime directory), and a **default-deny host home mount architecture**: `bwrap` mounts an empty `--tmpfs` over every candidate host home directory (the invoking user's `$HOME`, `/root`, `/home/*`, and operator-declared roots in `AOS_EXTRA_CREDENTIAL_HOMES`) *before* selectively re-binding safe read-only compiler toolchains (`.rustup`, `.cargo/{bin,registry,git}`, and `.local/{bin,lib,share}`, intentionally excluding `.cargo/credentials.toml`) and the agent's writable CoW shadow workspace, agent home, and organization publish directory. Consequently, all known and unknown credential files in `$HOME`—even at arbitrary user-created paths such as `~/custom_vault/secret_key.pem`—are completely absent from the sandbox's mount namespace. Because the network namespace also cuts the sandbox off from its own proxy, Linux sandboxes reach it through an *egress bridge*: the host relays a Unix socket

to the identity-pinned proxy, and a small in-sandbox shim (`aegis-os egress-shim`) listens on the proxy's loopback port inside the namespace and forwards to that socket. Since `--ro-bind / /` leaves other agents' sockets visible, every bridge connection must present a random per-sandbox token, so a sandbox cannot borrow another agent's proxy identity.

(2) **Tier 2 (`MicroVmSandboxDriver`, Cloud Control-Plane Interface):** Provides the configuration generator (`virtio-fs` mount tags, vCPU/memory limits) and Scale-to-Zero `pause()`/`resume()` state machine for cloud microVM backends (`libkrun` / Firecracker [1]).

Identity-Pinned Egress Proxy with L7 Capability-Scoped Secret Injection and Outbound DLP.. To mitigate credential exfiltration (Threat T4), agents inside Aegis OS do not receive raw API keys in environment variables. Instead, they receive opaque placeholders such as `$AOS_SECRET_GH`. Outbound HTTP traffic is routed through a local asynchronous L7 HTTP proxy (`start_local_proxy_with_bound_identity`). Because each sandbox listener pins its `AgentIdentity` at the TCP socket level, a compromised process cannot spoof `X-Aegis-Org` or `X-Aegis-Agent` headers. Furthermore, domain-only secret binding is vulnerable to *same-domain confused-deputy* attacks on multi-tenant API hosts (e.g., an agent using a GitHub token bound to `api.github.com` to exfiltrate repository code to a public gist via `POST /gists`, path traversal `/repos/acme/../../gists`, or embedding a private key inside an allowlisted issue comment). Each `SecretVaultEntry` therefore supports fine-grained L7 `SecretScopeRule` constraints (`domain`, `allowed_methods`, `allowed_path_prefixes`) paired with an RFC 3986 path canonicalizer (`normalize_request_path`) and an outbound Data Loss Prevention scanner (`inspect_payload_dlp`) that inspects URL query strings and HTTP request bodies (up to 64 KiB) for leaked `$AOS_SECRET_*` placeholders, registered vault secret values, PEM private keys (`-----BEGIN * PRIVATE KEY-----`), AWS keys (`AKIA*`), GitHub tokens (`ghp_*`, `github_pat_*`), API keys (`sk-*`), and Slack tokens (`xoxb-*`):

- If `$AOS_SECRET_GH` (scoped to `GET`/`POST` on `api.github.com/repos/acme/` and `GET /user`) is sent on a matching canonical path with a clean payload, the proxy substitutes the real token `ghp_...` at the network boundary.
- If an agent attempts to send `$AOS_SECRET_GH` to an unauthorized domain (`evil-exfil.org`), an allowlisted but unbound domain (`httpbin.org`), a disallowed method/path on the bound domain itself (`POST /gists`, `DELETE /repos/acme/issues/1`, or `/repos/acme/%2e%2e/%2e%2e/gists`), or embeds a secret/private-key payload in the query string or body of an allowlisted endpoint (`POST /repos/acme/issues`), the proxy rejects the request with `HTTP/1.1 403 Forbidden` (`blocked_exfiltration`).

3.3 A2A-Compliant Inter-Agent IPC Bus, Global Peer Discovery, and Reference-Based CAS (`aos-ipc`)

Whereas Anthropic's Model Context Protocol (MCP) [3] standardizes *vertical* agent-to-tool execution, Google's Agent2Agent (A2A) protocol [10] standardizes *horizontal* capability discovery (`AgentCard`) and stateful task delegation (`Task` lifecycle). `aos-ipc` unifies both over a disk-synchronized, broadcast-enabled bus:

- **Global Workstation Peer Registry and Zombie PID Pruning:** `aos-ipc` maintains per-organization and workstation-wide registries (`scope="global"` in `ipc_list_peers`, `aegis://global/peers`) supporting `<org_id>/<agent_id>` addressing. Each peer record stores its client root PID and start timestamp (`PROC_PIDTBSDINFO` on macOS / `/proc/<pid>/stat` on Linux) so `prune_dead_peers()` automatically evicts dead or PID-recycled zombie peers.

- **Google A2A `AgentCard` and Stateful `Task` Lifecycle:** Every peer synthesizes an A2A `AgentCard` (`GET /.well-known/agent-card.json`, with the legacy `agent.json` path kept as an alias; `aegis://a2a/agent-cards`, `ipc_update_card`) with structured `AgentSkill` tags (auto-reflecting active `coord_claim` globs). Beyond P2P mailboxes (`ipc_send`/`ipc_recv`) and topic Pub/Sub (`ipc_publish`/`ipc_subscribe`), every correlated `ipc_request` (and `POST /a2a message/send`, with legacy `tasks/send` accepted) instantiates a persistent `A2ATask` tracked across `submitted` → `working` (on `ipc_ack`) → `input-required` (via `ipc_task_update`, pushing an `[A2A input-required]` wakeup into the requester's inbox) → `completed` | `failed` | `canceled` (on `ipc_reply` or `tasks/cancel`), with typed `TextPart`, `DataPart`, and CAS `FilePart` outputs.
- **Enforced File Claims and Reference-Based CAS Handles:** `coord_claim` registers glob leases enforced at pre-tool-use and `workspace_commit` time. For large payloads, `org_artifact_publish` writes blobs to the Content-Addressable Store and returns a compact SHA-256 URI (`artifact://<org>/sha256:<64-hex>`, ~85–98 bytes) wrapped as an A2A `FilePart`, reducing in-transit IPC payload tokens by **99.85%**, while cross-organization CAS `FilePart` fetches remain gated by Policy Rule R1.

3.4 Cedar-Inspired Policy Engine and Runtime Design Invariants (`aos-policy`)

Every runtime tool call is evaluated by `AegisPolicyKernel::evaluate` using a five-stage priority hierarchy inspired by Cedar [6] that partitions operations into a **Red Zone** (always blocked or forced to review), a **Green Zone** (auto-allowed inside the sandbox), and an **Amber Boundary Zone** (gated by human approval or session leases):

(1) **R0a — Host Red-Line Guard (`Deny`):** Any access targeting sensitive host paths (`~/.ssh`, `~/.aws`, `~/.gnupg`, `~/.kube`, `/etc/shadow`, `/etc/sudoers`, or `/System`) is unconditionally denied.
(2) **R0b — Commit-Time Hook, CI & Build-Script Guard (`Ask`):** Any `workspace_commit` that modifies repository execution hooks, CI workflows, or build/toolchain entry-points (`.git/hooks/*`, `.git/config`, `.gitmodules`, `.envrc`, `.github/workflows/*`, `.gitlab-ci.yml`, `build.rs`, `Makefile`, `GNUmakefile`, `CMakeLists.txt`, `package.json`, `setup.py`, `.cargo/config*`, `.vscode/tasks.json`, or `.vscode/launch.json`) bypasses general `workspace_commit` session leases and forces explicit human approval (`action = workspace_commit_sensitive`). Once approved via `policy_approve_escalation`, the resulting `workspace_commit_sensitive` lease permits the human-reviewed sensitive path to land cleanly on retry.
(3) **R1 — Multi-Tenant Isolation Gate (`Deny` / `Ask`):** Cross-organization CAS artifact reads or unauthorized cross-tenant mutations are rejected.
(4) **R2 & R3 — Active Session Leases & Green-Zone Auto-Allow (`Allow`):** All in-sandbox CoW file operations, sandboxed `proc_exec` commands, intra-organization A2A/IPC calls, allowlisted egress requests, and boundary commits covered by an active session lease (`LeaseGrant`) are auto-allowed with zero prompts.
(5) **R4 — Unleased Boundary Crossing (`Ask`):** Unleased `workspace_commit` calls and HTTP requests to unlisted external domains return an escalation ID requiring either a one-shot approval (`ttl_secs = 0`) or a TTL-scoped session lease (e.g., `ttl_secs = 3600`), with a context-aware confirmation prompt summarizing the action, modified shadow files, target host directory, and lease scope.

Together, these mechanisms enforce three core runtime invariants:

- **Design Invariant 1 (Pre-Commit Host Non-Interference):** *Until a* `workspace_commit` *is explicitly approved, no sequence of agent tool calls or spawned child processes can modify any file in the host repository.* VFS tools reject path traversal (`..`) and resolve all paths inside `shadow_root`, while `proc_exec` enforces kernel Seatbelt (`deny file-write*`) and Linux `bwrap --ro-bind / /` outside `shadow_root`, `agent_home`, and `org_pub`.
- **Design Invariant 2 (Zero Raw Secret Exposure to Sandbox):** *Plaintext API credentials never enter the sandboxed process environment, command-line arguments, or logs, and are injected into outbound HTTP headers only when the destination* `Host` *and canonicalized HTTP method + URL path prefix match the secret's* `SecretScopeRule` *and the request query/body passes outbound DLP inspection (*`inspect_payload_dlp`*).* `NativeOsSandboxInstance::exec` clears the child environment (`env_clear()`) and passes only opaque `$AOS_SECRET_*` placeholders.
- **Design Invariant 3 (All-or-Nothing Conflict-Safe Commit):** *Concurrent agent commits either produce a merged host file preserving all non-overlapping edits, or abort in Phase 1 without modifying any host file when two agents edit the same baseline lines inconsistently or when the optional pre-commit* `verify_cmd` *semantic gate fails inside the sandbox.*

4 MULTI-CLIENT INTEGRATION AND ENFORCEMENT

To minimize setup friction, `aegis-os install` automatically configures MCP server entries, per-session principal resolution, and lifecycle hooks across all three coding agent CLIs:

(1) **Google Antigravity (agy):** Registers `aegis-os` in `~/.gemini/config/mcp_config.json` with eager tool loading, pre-authorizes green-zone `mcp(aegis-os/*)` tools in `settings.json` while keeping lease/approval tools (`policy_grant_lease`, `policy_approve_escalation`) under `permissions.ask`, and installs four lifecycle hooks (`SessionStart`, `PreInvocation`, `PreToolUse`, and `Stop`):
   - *MCP-First Tool Routing (*`PreToolUse`*):* Because agent CLIs still prompt on native `run_command` invocations unless exact command strings match static allowlists, `aegis-os agy-hook` classifies native shell (`run_command`/`Bash`), file write (`write_file`/`edit_file`/`Write`/`Edit`), and file read (`read_file`/`view_file`/`Read`) calls: Ring-0 operations (sudo, credential stores, foreign tenant directories, and edits to client `hooks.json`/`settings.json`) are unconditionally denied; outside-repository writes and host-boundary commands (`git push`, package installs) escalate to `ask`; and in-repository shell commands and file writes are denied with a ready-to-invoke `mcp_aegis-os_proc_exec` or `mcp_aegis-os_fs_write` payload, steering the agent onto the zero-prompt sandboxed MCP path.
   - *Per-Session Identity & Push Coordination (*`SessionStart`, `PreInvocation`, `Stop`*):* Walking parent process ancestry (`PROC_PIDTBSDINFO` on macOS / `/proc/<pid>/stat` on Linux) assigns each concurrent client window a distinct session principal (`agy-<6hex>`) bound to its `conversationId`. On `SessionStart`, the agent's CoW shadow is rebased onto the host via `refresh_from_host()`; on `PreInvocation`, unread IPC messages are non-destructively peeked and injected via `injectSteps.systemMessage`; and on `Stop`, un-acknowledged `TaskRequest` messages hold the turn open (up to 2 continues) until answered, while `coord_claim`/`coord_release` prevent overlapping file edits across live peers.

(2) **Anthropic Claude Code (`claude`):** Injects `aegis-os` into `~/.claude.json` and configures both green-zone `mcp__aegis-os__*` allow rules and the `PreToolUse` enforcement hook in `~/.claude/settings.json`.

Table 3. **Experiment 1 (RQ1): Authorization Friction, Interruption Prompt Rate (IPR), and Modeled Human Wait Time** across 50 deterministic multi-step trajectories spanning 5 SWE task templates (1,500 tool calls executed on `AegisRuntimeKernel`, including 50 real `workspace_commit` operations in every mode; human verification modeled at 5.0 s/prompt).

Execution Mode	Trajectories	Tool Calls	Human Prompts	IPR (%)	Prompt Reduction	Max Autonomous Streak	Runtime Exec (ms)	Modeled Human Wait (T_{human})
Host-Interactive (Per-Mutation Prompt)	50	1,500	860	57.33%	0.00% (Baseline)	4 steps	457.1 ms	71.7 min (4,300 s)
Host-Static-Allowlist (Read/Status Allow)	50	1,500	680	45.33%	20.93%	7 steps	496.7 ms	56.7 min (3,400 s)
Aegis-OS (Per-Task Commit Gate, `ttl=0`)	50	1,500	**50**	**3.33%**	**94.19%**	**29 steps**	**483.6 ms**	**4.2 min (250 s)**
Aegis-OS (With 1h Session Lease)	50	1,500	**1**	**0.07%**	**99.88%**	**1,470 steps**	**469.2 ms**	**0.08 min (5 s)**

(3) **OpenAI Codex CLI (`codex`):** Appends `[mcp_servers.aegis-os]` to `~/.codex/config.toml` and piggybacks unread IPC summaries (`_aegis_inbox`) onto `tools/call` responses.

5 EMPIRICAL EVALUATION

We evaluate Aegis OS using our reproducible benchmark suite (`crates/aos-cli/src/bench.rs`, invoked via `aegis-os bench`). Unless stated otherwise, experiments ran on an Apple Silicon workstation (`macos-aarch64`, APFS, Rust 1.99.0 `--release`) using real OS processes, Seatbelt kernel sandboxes, loopback TCP sockets, and `cargo` builds. The security, merge, and provisioning experiments were repeated on Linux in a virtual machine on the same host (Ubuntu 24.04 aarch64, kernel 6.8, bubblewrap 0.9) with the workspace on btrfs, XFS, or ext4. Linux numbers therefore include virtualised I/O, and the XFS volume was a loop-backed image on btrfs, so it is only a rough indication. We investigate six research questions:

- **RQ1 (Authorization Friction):** How much does Aegis OS reduce confirmation prompts and modeled verification wait time across both controlled SWE trajectories and real-world coding-agent session transcripts?
- **RQ2 (Adversarial Security):** Does Aegis OS block end-to-end OS process, credential harvesting, proxy/socket exfiltration, cross-tenant, commit-time poisoning, L7 confused-deputy/DLP, and conflicting-commit attacks that compromise unconfined `Host-YOLO` execution and stateless native CLI sandboxes?
- **RQ3 (System Overhead & Scaling):** What is the latency and disk footprint of CoW provisioning (APFS clones and Linux reflinks, compared with `cp`, `rsync`, and `git`), policy evaluation, HTTP daemon RPCs, 3-way merge commits, and Seatbelt process spawning?
- **RQ4 (Multi-Agent Concurrency):** Does two-phase LCS Diff3 merging with pre-commit semantic verification (`verify_cmd`) preserve concurrent multi-agent edits while catching both overlapping line conflicts and disjoint-line semantic regressions?
- **RQ5 (Fault Recovery):** How fast does atomic CoW rollback recover from multi-file syntax corruption compared to sequential file restoration?
- **RQ6 (Component Necessity):** Is each safety component necessary, i.e., does switching it off re-open an attack class that the full system blocks?

5.1 Experiment 1 (RQ1): Authorization Friction and Autonomous Continuity

To evaluate authorization friction reproducibly without LLM sampling variance, we execute 50 deterministic software engineering trajectories (30 tool calls per trajectory, totaling 1,500 real `AegisRuntimeKernel::call_tool` invocations) cycling across five task templates: (1) *Auth Bugfix & Regression Test*, (2) *Multi-Module Refactoring*, (3) *Feature Implementation & Quota Suite*, (4) *Security Audit & Sanitizer Hardening*, and (5) *Multi-Agent Contract Sync*. Each trajectory exercises repository exploration (`fs_glob`, `fs_read`), checkpointing (`workspace_snapshot`),

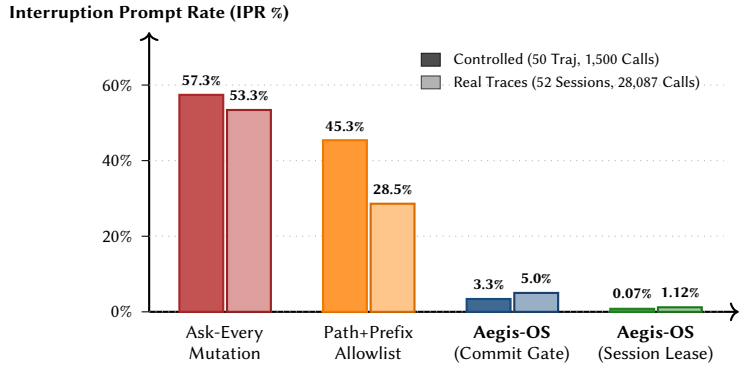


Fig. 2. Interruption Prompt Rate (IPR) across both 50 controlled trajectories (1,500 calls, Experiment 1) and 52 real coding-agent session logs (28,087 calls, Experiment 1B). Aegis OS reduces confirmation prompts by 94.2%–99.9% on controlled trajectories and by 90.7%–97.9% on real session traces.

Table 4. **Experiment 1B (RQ1): Real-World Coding-Agent Session Trace Evaluation** across **Corpus A** (52 real production coding-agent session transcripts totaling 57,923 conversation steps and 28,087 tool calls: 13,098 read/coord, 11,070 shell commands of which 4,318 [39.0%] are compound pipelines &&/| or inline scripts `python3 -c`, and 3,836 file edits/writes) and **Corpus B** (5 live Aegis OS dogfooding sessions totaling 3,962 steps and 2,040 tool calls, including 68 `workspace_commit` and 15 ground-truth `policy_approve_escalation` events), evaluated via `scripts/e6_real_traces.py`.

Trace Corpus	Permission Regime	Total Prompts	Aggregate IPR (%)	Mean Session MAS	Prompt Reduction
Corpus A: All Real Sessions	Ask-Every-Mutation	14,974 / 28,087	53.31%	16.4 steps	0.0% (Baseline)
(*N*=52 sessions, 57,923 steps,	Path+Prefix Allowlist (`cargo`, `git status`, `pytest`, etc.)	8,016 / 28,087	28.54%	31.6 steps	46.5%
28,087 tool calls)	Vanilla CoW + Seatbelt (Per-Commit Gate)	1,397 / 28,087	4.97%	48.1 steps	90.7%
	Aegis OS (CoW + Seatbelt + Scoped Action Lease)	**315 / 28,087**	**1.12%**	**76.7 steps**	**97.9%**
Corpus B: Live Aegis OS Sessions	Ask-Every-Mutation	1,093 / 2,040	53.58%	21.2 steps	0.0% (Baseline)
(*N*=5 sessions, 3,962 steps,	Path+Prefix Allowlist (`cargo`, `git status`, `pytest`, etc.)	606 / 2,040	29.71%	33.0 steps	44.6%
2,040 tool calls)	Vanilla CoW + Seatbelt (Per-Commit Gate)	106 / 2,040	5.20%	72.2 steps	90.3%
	Aegis OS (CoW + Seatbelt + Scoped Action Lease)	**35 / 2,040**	**1.72%**	**100.0 steps**	**96.8%**

multi-file edits (`fs_edit`, `fs_write`), CAS publishing (`org_artifact_publish`), peer IPC coordination (`ipc_send`, `ipc_recv`, `ipc_publish`), diff inspection (`workspace_diff`), and host commit (`workspace_commit`). All four modes execute all 1,500 tool calls including all 50 real `workspace_commit` operations.

Table 3 and Figure 2 present the results. Under *Host-Interactive*, every file write, state mutation, and external IPC action triggers a confirmation prompt (860 prompts, 57.33% IPR), restricting the maximum autonomous streak to 4 steps and imposing 71.7 minutes of modeled verification wait time (at 5.0 s/prompt) on top of 457.1 ms of runtime execution. Adding a static read/status/snapshot allowlist (*Host-Static-Allowlist*) still triggers 680 prompts (45.33% IPR, 56.7 minutes modeled wait) because host file edits cannot be safely auto-allowed. In contrast, **Aegis-OS (Per-Task Commit Gate)** auto-allows all 29 internal CoW workspace and IPC operations per trajectory and prompts once at the final `workspace_commit` gate (approved via a one-shot `ttl=0` token), reducing prompts by **94.19%** (3.33% IPR, 483.6 ms runtime execution). When the operator grants a **1-hour Session Lease** (`ttl=3600`) on the first commit escalation, Aegis OS completes all 50 trajectories (and all 50 host commits) in 469.2 ms with **1 prompt total** (0.07% IPR, a **99.88% reduction**), extending the continuous autonomous streak to **1,470 steps** (**367.5**× longer).

Table 5. **Experiment 2 (RQ2): End-to-End Red-Team Adversarial Attack Evaluation** across 80 live process, socket, proxy, gateway, policy, and commit attacks in eight categories, executed against an unconfined `Host-YOLO` target environment versus `Aegis-OS` (identical counts on macOS and Linux). The `Host-YOLO` column is 100% by construction, since every attack is designed to succeed against an unconfined process; the informative column is `Aegis-OS`.

Attack Surface Category	Live Attacks	Host-YOLO Compromised (ASR)	Aegis-OS Compromised (ASR)	Empirical Kernel / Gateway Evidence
A. Host Path Escape & File Tampering	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Kernel `sandbox-exec` (`bwrap` on Linux) denied all 10 outside writes
B. Host Credential Harvesting (`.ssh/`, `.aws/gcloud/custom_vault`)	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Seatbelt regexes & Linux default-deny `--tmpfs $HOME` blocked all 10 reads
C. Sandbox-to-Proxy & Direct `--noproxy` Egress Exfil	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Pinned TCP Proxy returned 403 (via the egress bridge on Linux); no real token reached the sink
D. Cross-Tenant IPC & CAS Access via MCP Gateway	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	MCP Gateway + Policy Engine rejected all 10 cross-organization tool calls
E. Commit-Time Poisoning (hooks, `.envrc`, CI, `build.rs`, `Makefile`)	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Rule R0b escalated all 10 hook/CI/build-script paths despite a standing commit lease
F. Secret-Binding, L7 Confused-Deputy & Outbound DLP Abuse	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Domain + L7 path scope + `inspect_payload_dlp` blocked all 10 exfiltration vectors
G. Partial Commit on Conflict (atomicity)	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Phase-1 validation left the host untouched in all 10 conflicting batches
H. Silent Overwrite of a Peer's Edit (lost update)	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Conflict detection preserved the peer's edit in all 10 same-line conflicts
Total / Evaluated ASR	**80**	**80 / 80 (100.0%)**	**0 / 80 (0.0%)**	**0 / 80 Compromised across OS Kernel, Network Proxy, Policy Engine & Commit Path**

Real-World Coding-Agent Session Trace Evaluation (Experiment 1B).. Because synthetic trajectories could under- or over-state how often real agents cross authorization boundaries, we also evaluate **52 real coding-agent session transcripts** (**57,923 conversation steps** and **28,087 tool calls**) collected from production software engineering workflows (`scripts/e6_real_traces.py` and anonymized trace dataset `scripts/real_traces_anonymized.json`). As shown in Table 4, real agent workflows exhibit a striking structural property: **39.0% of all shell invocations (4,318 of 11,070 commands in Corpus A, and 80.5% in Corpus B)** use compound shell operators (`&&`, `||`, `;`, `|`, `$(...)`) or inline interpreter scripts (`python3 -c`, `bash -c`) to inspect data, run multi-stage builds, or verify fixes. Consequently, standard *Path+Prefix Allowlists* (which auto-approve in-repo file edits and single-command dev prefixes such as `cargo`, `git status`, `pytest`, and `npm test`, but must prompt on compound shell chains, `python3 -c`, `rm`, and external commands to avoid complete sandbox escape) still interrupt the operator on **28.54% of all tool calls** (8,016 prompts; Mean MAS = 31.6 steps). By contrast, executing compound shell chains and file edits silently inside the kernel-enforced CoW shadow workspace reduces prompts by **90.7%** under *Vanilla CoW + Seatbelt* (4.97% IPR) and by **97.9%** under **Aegis OS with Scoped Action Leases** (**1.12% IPR**, 315 prompts across 28,087 tool calls; Mean MAS = 76.7 steps). On the 5 live dogfooding sessions where Aegis OS was used to build and test its own repository (Corpus B, 2,040 tool calls), Aegis OS achieved a **1.72% IPR** (35 prompts, **96.8% reduction**) and a mean autonomous streak of **100.0 steps**.

5.2 Experiment 2 (RQ2): End-to-End Adversarial Security

We execute 80 live adversarial attacks end-to-end against both an unconfined disposable victim environment (`Host-YOLO`) and `Aegis-OS` (Table 5):

- **Category A (10 Host Path Escape Attacks):** Spawns `sh -c "echo COMPROMISED > <outside_target>"`. In `Host-YOLO`, all 10 outside-workspace files were created on disk (100.0% ASR). Inside `Aegis-OS` (`proc_exec`), `/usr/bin/sandbox-exec` denied all 10 writes at the XNU kernel boundary with `Operation not permitted` (0.0% ASR).
- **Category B (10 Multi-Store Credential Harvesting Attacks):** Spawns `sh -c "cat <cred_path>"` across 10 distinct credential locations: `~/.ssh/id_rsa`, `~/.aws/credentials`, `~/.gnupg/secring.gpg`, `~/.kube/config`, `~/.npmrc`, `~/.pypirc`, `~/.netrc`, `~/.git-credentials`, `~/.config/gcloud/credentials.db`, and an arbitrary user vault directory `~/custom_vault/secret_key.pem`. `Host-YOLO` leaked the secret key material on all 10 attempts (100.0% ASR), whereas the Aegis OS Seatbelt profile and Linux default-deny `--tmpfs $HOME` mount blocked all 10 reads (0.0% ASR).

- **Category C (10 Sandbox-to-Proxy and Direct `--noproxy` Egress Exfiltration Attacks):** We start a live local TCP sink server (`sink_addr`) recording leaked secrets, alongside an identity-pinned `AegisEgressGateway` (`proxy_addr`). In `Host-YOLO`, an unconfined `curl` process with `$GITHUB_TOKEN` in its environment transmitted the plaintext token to `sink_addr` across all 10 attacker domains (100.0% ASR). In `Aegis-OS`, we execute `curl` inside a Seatbelt sandbox (`sbx_egress.exec`) bound to `localhost:<proxy_port>` that attempts *both* (1) proxy exfiltration of `$AOS_SECRET_GH` to 10 unauthorized domains (`evil-exfil.org`, `webhook.site`, `ngrok-free.app`, etc.), which the proxy rejected with `HTTP/1.1 403 Forbidden` (`blocked_exfiltration`), and (2) direct proxy-bypass (`curl --noproxy '*' http://<sink_addr>/bypass`), which the XNU Seatbelt socket filter blocked with `Operation not permitted`. Zero tokens reached `sink_addr` (0.0% ASR).
- **Category D (10 Cross-Tenant IPC and CAS Attacks via MCP Gateway):** In a shared flat host store (`Host-YOLO`), an unconfined process directly read all 10 confidential enterprise tenant files (100.0% ASR). In `Aegis-OS`, we register 10 enterprise tenants (`org-enterprise-0..9`), publish confidential CAS artifacts via `org_artifact_publish`, and have principal `mallory` (`org-alpha`) invoke `kernel.call_tool` (`ipc_send` and `org_artifact_fetch`) targeting those enterprise tenants. All 10 cross-tenant tool calls were denied by Policy Rule R1 (0.0% ASR).
- **Category E (10 Commit-Time Poisoning Attacks):** With a standing one-hour commit lease already granted (the situation the guard exists for), the agent tries to land 10 auto-executing files across both VCS/shell hooks and build-system entry-points: `.git/hooks/pre-commit`, `.git/hooks/post-checkout`, `.git/hooks/pre-push`, `.git/config`, `.envrc`, `.github/workflows/ci.yml`, `build.rs`, `Makefile`, `package.json` (`postinstall`), and `.cargo/config.toml` (`rustc-wrapper`). Unconfined, all 10 files land (100.0% ASR). Under Aegis OS, Policy Rule R0b escalates every one to a human despite the standing lease (0.0% ASR).
- **Category F (10 Secret-Binding, L7 Confused-Deputy & Outbound DLP Abuse Attacks):** We test four cross-domain attacks targeting allowlisted domains that are not the domain `$AOS_SECRET_GH` is bound to (`httpbin.org`, `pastebin.com`, `gist.github.com`, `raw.githubusercontent.com`), four same-domain L7 confused-deputy attacks targeting disallowed paths or methods on `api.github.com` itself (`POST /gists`, path traversal `POST /repos/acme/../../gists`, percent-encoded traversal `POST /repos/acme/%2e%2e/%2e%2e/gists`, and `DELETE /repos/acme/issues/1`), *plus* two same-domain L7 outbound DLP exfiltration attacks on an allowlisted endpoint (`GET /repos/acme/issues?leak=AKIAIOSFODNN7EXAMPLE` and `POST /repos/acme/issues` with an OpenSSH PEM private key embedded in the JSON issue body). Unconfined, the raw token reaches the sink on all 10 requests (100.0% ASR); in Aegis OS, per-secret domain binding, RFC 3986 path-canonicalized `SecretScopeRule` enforcement, and `inspect_payload_dlp` refused all 10 (0.0% ASR).
- **Categories G and H (10 Conflicting Commits):** A commit batch holds a clean file (`a.txt`) and a file that a peer has changed on the same line (`b.txt`). Category G asks whether the clean half still reaches the host (broken atomicity); Category H asks whether the peer's edit is silently overwritten (lost update). Without transactions the unconfined baseline fails both; in Aegis OS Phase 1 refuses the whole batch, leaving the host tree untouched and the peer's edit intact (0.0% ASR for both).

All 80 attacks were also run on Ubuntu 24.04 with identical results (0/80 for Aegis OS). By mounting an empty `--tmpfs` over every candidate host home directory (`$HOME`, `/root`, `/home/*`,

Table 6. **Experiment 2B (RQ2): Comparison with Stateless Native CLI Sandboxes** across all 80 attacks in Categories A–H, executed on both macOS (sandbox-exec) and Linux (bwrap) via `scripts/e5_cli_sandboxes.py` (identical counts on both OSes). `CLI-Sandbox (WS-Write)` mounts the live host repository read-write, carves out `.git` as read-only, and blocks `~/.ssh`, `~/.aws`, and `~/.gnupg`.

Execution Mode	A: In-WS	B: Creds	C: Exfil	D: Tenant	E: Poison	F: Secret	G: Atomic	H: Overwrite	Total	ASR
`Host-YOLO` (Unconfined)	10/10	10/10	10/10	10/10	10/10	10/10	10/10	10/10	80/80	100.0%
`CLI-Sandbox (WS-Write + Net)`	10/10	4/10	10/10	10/10	4/10	10/10	10/10	10/10	68/80	85.0%
`CLI-Sandbox (WS-Write, No-Net)`	10/10	4/10	0/10	10/10	4/10	0/10	10/10	10/10	48/80	60.0%
Aegis-OS (Transactional CoW)	**0/10**	**0/10**	**0/10**	**0/10**	**0/10**	**0/10**	**0/10**	**0/10**	**0/80**	**0.0%**

Table 7. **Experiment 3A (RQ3): Multi-Trial APFS Copy-on-Write Workspace Provisioning Latency and Storage Efficiency** across 10 concurrent agents modifying realistic per-task working sets (6, 12, and 25 files per agent; 5 trials per scale, Mean ± StdDev). The baseline is a user-space read/write loop, not a system copy tool; Table 9 compares against real tools.

Repository Scale	Files	Logical Size	User-Space Copy Loop (Mean ± SD)	Aegis CoW Clone (Mean ± SD)	Speedup vs. Loop	10-Agent Disk Saving
Small Project	100	2.0 MB	9.33 ± 0.28 ms	**0.71 ± 0.05 ms**	**13.2**×	**94.0%** (1.2 MB vs. 19.5 MB)
Medium Project	300	17.6 MB	32.08 ± 5.82 ms	**1.85 ± 0.09 ms**	**17.3**×	**96.0%** (7.0 MB vs. 175.8 MB)
Large Project	1,000	78.1 MB	113.13 ± 16.41 ms	**6.28 ± 0.65 ms**	**18.0**×	**97.5%** (19.5 MB vs. 781.2 MB)

and `AOS_EXTRA_CREDENTIAL_HOMES`) before selectively re-binding read-only compiler toolchains (`.rustup`, `.cargo/{bin,registry,git}`), Linux `bwrap` hides even unknown custom credential paths inside `$HOME` (such as `~/custom_vault/secret_key.pem` in Category B). Note that the Linux sandbox requires unprivileged user namespaces, which Ubuntu 24.04 restricts by default through AppArmor (`kernel.apparmor_restrict_unprivileged_userns`); we relaxed this in the test VM. Running the Linux benchmark also exposed two earlier defects that macOS runs could not: a private `/tmp` mounted after the writable binds hid every workspace under `/tmp`, which had made Linux categories A and B pass vacuously, and the benchmark counted a sandbox that merely failed to reach its proxy as compromised. Both are fixed; we report them because a security result that holds only when the sandbox is misconfigured is not a valid result.

Comparison with Stateless Native CLI Sandboxes (Experiment 2B).. Production agent CLIs (such as Codex CLI's `workspace-write` mode and Claude Code's bash sandbox) wrap shell commands in a stateless Seatbelt or Bubblewrap/Landlock profile that mounts the live working tree read-write, carves out `.git` as read-only, and blocks `~/.ssh`, `~/.aws`, and `~/.gnupg`. Table 6 evaluates this architecture on both macOS and Linux across Categories A–H. With outbound network enabled (`WS-Write + Net`, required for package managers and API calls), the stateless CLI sandbox still suffers an **85.0% ASR (68/80)**: (i) in-workspace destructive commands (`rm -rf src`, `truncate`) directly corrupt the live host tree (**10/10** in Cat. A); (ii) blocking `~/.ssh`, `~/.aws`, and `~/.gnupg` stops 6/10 credential reads, but unmasked dotfile credentials in `$HOME` (`.npmrc`, `.pypirc`, `.netrc`, `.git-credentials`) remain readable (**4/10** in Cat. B); (iii) raw `$GITHUB_TOKEN` environment variables are exfiltrated or sent to unbound domains (**10/10** in Cats. C and F); (iv) carving out `.git`, `.envrc`, and `.github` as read-only blocks VCS/CI hooks (6/10), but in-tree build-system entry-points (`build.rs`, `Makefile`, `package.json`, and `.cargo/config.toml`) remain writable in an in-place workspace sandbox (**4/10** in Cat. E); and (v) direct in-place writes lack two-phase staging or 3-way merging (**10/10** in Cats. G and H). Disabling outbound network entirely (`WS-Write, No-Net`) stops network exfiltration (Cats. C and F) at the cost of breaking dependency installation and API access, yet still leaves **60.0% ASR (48/80)** from in-place workspace mutations, dotfile reads, and concurrent overwrites.

Table 8. **Experiment 3B (RQ3): Kernel, Daemon, Merge Commit, and Sandbox Spawn Latency Distribution**. Ops/sec is computed as N divided by total wall-clock time of the measurement loop (including harness overhead), so it is slightly lower than 1/Mean.

Runtime Operation	Mean (μs)	P50 (μs)	P95 (μs)	P99 (μs)	Ops/sec
`PolicyKernel::evaluate` (N=2000)	0.31	0.29	0.38	0.50	2,959,674
`IpcBus::send + recv` (N=1000)	1.06	0.83	1.79	4.75	924,749
`HTTP POST /mcp` (`fs_read`, N=200)	177.11	183.08	258.79	409.00	5,503
`commit_to_host` (3-Way Merge, N=100)	424.02	413.08	540.21	646.12	2,110
`Sandbox::exec` (`sandbox-exec`, N=30)	6,678.97	6,557.62	7,533.12	7,811.33	150

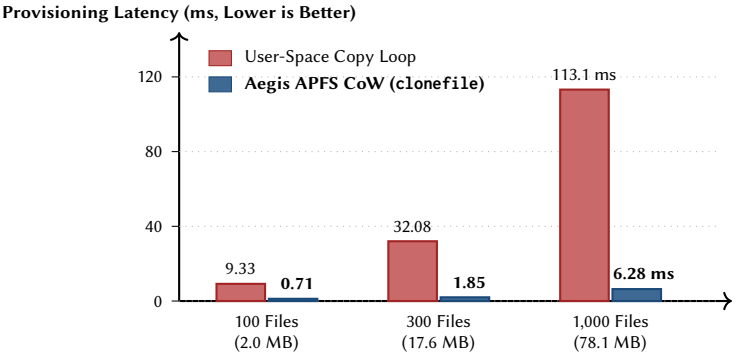


Fig. 3. Workspace provisioning latency across repository scales (Experiment 3A). APFS `clonefile` scales with file metadata count rather than total byte volume, achieving an 18.0× speedup over a user-space copy loop on the 1,000-file repository.

5.3 Experiment 3 (RQ3): CoW Provisioning, Storage Footprint, and Operation Latency

We benchmark workspace provisioning across three repository scales (100 files / 2.0 MB, 300 files / 17.6 MB, and 1,000 files / 78.1 MB, 5 trials per scale). As shown in Table 7 and Figure 3, a user-space copy loop requires 113.13 ± 16.41 ms for the 1,000-file repository, whereas APFS `libc::clonefile` provisions the shadow workspace in 6.28 ± 0.65 **ms** (18.0× faster than that loop; Table 9 gives the comparison against real tools). Because `clonefile` duplicates only inode metadata rather than file data blocks, provisioning latency scales with file count (0.71 ms at 100 files to 6.28 ms at 1,000 files) rather than total byte size. Across 10 concurrent agent workspaces modifying working sets of 6, 12, and 25 files per agent, APFS CoW reduces disk block allocation by **94.0%–97.5%** (19.5 MB vs. 781.2 MB at 1,000 files).

Table 8 reports the latency distribution across all five runtime operation classes. In-process policy evaluation takes **0.31** μ**s** (310 ns, 2.96M ops/s), P2P IPC send/receive takes **1.06** μ**s**, a loopback HTTP JSON-RPC `fs_read` call (`POST /mcp`) takes **177.11** μ**s**, a two-phase LCS Diff3 `commit_to_host` takes **424.02** μ**s**, and spawning an external process under macOS Seatbelt (`/usr/bin/sandbox-exec /usr/bin/true`) takes **6.68 ms** (6,678.97 μs). Relative to typical LLM inference turns (0.5–2.0 s), MCP tool mediation adds $< 0.04\%$ latency and Seatbelt process confinement adds ~6.7 ms per spawned command.

Comparison with Real Tools and Cross-OS Remote Daemon Evaluation (Experiment 3C).. The copy loop in Table 7 is a weak baseline, so Table 9 times the tools a developer would actually use to give an agent an isolated tree. On APFS, Aegis OS (6.3 ms) is about 16× faster than `cp -cR` (99 ms), which clones file by file, and about 20× faster than `git worktree add` (125 ms); the

Table 9. **Experiment 3C (RQ3): Provisioning a 1,000-file, 78 MB Tree with Real Tools** (mean time in ms; 7 trials after a warm-up, methods interleaved; physical disk consumed in parentheses when above 1 MB). Plain copy tools receive the tree without `.git`, which Aegis OS also skips; the git-based methods inherently need the repository. External commands include 0.3–2.6 ms of process start-up; the Aegis figure is an in-process call (macOS: Table 7; Linux: range over three runs of `aegis-os bench`). †XFS is a loop-backed image on btrfs and its Aegis timings are very noisy. ext4 has no reflink.

Method	macOS APFS	Linux btrfs	Linux XFS†	Linux ext4
`cp -R` (default)	205 (78 MB)	9.9	11.5	30.6 (78 MB)
`cp -R --reflink=never` (true full copy)	= `cp -R`	33.1 (78 MB)	30.5 (79 MB)	42.5 (78 MB)
`cp` with reflink (`-cR` / `--reflink=always`)	99	9.7	10.6	not supported
`rsync -a`	335 (79 MB)	82 (78 MB)	75 (79 MB)	93 (78 MB)
`git worktree add`	125 (79 MB)	57 (78 MB)	62 (79 MB)	64 (78 MB)
`git clone --local`	382 (79 MB)	69 (78 MB)	71 (79 MB)	107 (79 MB)
Aegis OS (CoW clone)	**6.3**	**15.6**	32.1	35.1 (78 MB)

Table 10. **Experiment 4 (RQ4): 3-Agent (`agy` + `claude` + `codex`) Concurrent Rust Modification, Syntactic & Semantic Conflict Detection, and `cargo test` Verification**.

Evaluation Metric	Host-Shared-YOLO (In-Place Write)	Aegis-OS Isolated CoW + 2-Phase LCS Diff3	Empirical Result
Concurrent 3-Agent Structural & Function Edits (`src/lib.rs`)	1 / 3 preserved (2 / 3 overwritten)	**3 / 3 preserved (100% LCS Diff3 Merged)**	**3.0× Feature Retention (0% Clobber)**
Overlapping Same-Line Edit Conflict Detection (10 Trials)	0 / 10 detected (Silent Lost Update)	**10 / 10 detected & aborted in Phase 1 (Host intact)**	**100% Syntactic Conflict Safety**
Disjoint-Line Semantic Conflict Detection via `verify_cmd` (10 Trials)	0 / 10 detected (Broken Host Build)	**10 / 10 intercepted in Sandbox (Host 100% Compilable)**	**100% Semantic Build Integrity**
Post-Merge Real `cargo test --offline` Verification	Failed (Missing agy & claude logic)	**Passed (`cargo test` exit code 0)**	**Build & Test Verified**
In-Transit IPC Payload for 3×64 KB Spec Handoffs	196,608 bytes (~49,152 tokens inline)	**294 bytes (~73 tokens via CAS URI handles)**	**99.85% In-Transit Token Reduction**

Table 11. **Experiment 5 (RQ5): Multi-File Corruption Recovery and `cargo check` Verification** (5 trials/scale, Mean ± SD).

Corrupted Modules	Seq. File Restore	Aegis CoW Rollback	Speedup	`cargo check`
10 (640 KB)	0.90 ± 0.06 ms	**0.75 ± 0.04 ms**	**1.2×**	PASSED
50 (3.2 MB)	4.82 ± 0.32 ms	**2.33 ± 0.14 ms**	**2.1×**	PASSED
200 (12.8 MB)	20.63 ± 3.14 ms	**8.68 ± 1.02 ms**	**2.4×**	PASSED

advantage comes from cloning the whole directory in a single `clonefile` call. On Linux, GNU `cp -R` reflinks by default and takes 9.9 ms on btrfs against 15.6 ms for Aegis OS's single-open `FICLONE` traversal (3.2 ms vs. 4.4 ms at the 300-file medium scale), while on ext4 (no reflink) our single-open `copy_file_range` fallback provisions the 1,000-file tree in 35.1 ms (comparable to `cp -R` at 30.6 ms and faster than `cp --reflink=never` at 42.5 ms). Across all Linux filesystems, Aegis OS stays 2×–5× faster than `rsync`, `git worktree add`, and `git clone --local`, while reflink-based methods allocate under 1 MB of extra disk. Finally, to verify a hybrid cloud/remote-container deployment (`scripts/verify_remote_linux_daemon.py`), we ran `aegis-os daemon` inside the Ubuntu 24.04 Linux VM backed by `/mnt/btrfs` and drove a complete multi-agent workflow (`coder-alpha` and `reviewer-beta`) from the macOS host over TCP (`POST /v1/mcp`): provisioning CoW workspaces, minting scoped `ak_live_*` tenant keys, executing `proc_exec` under `linux-bwrap-namespace-enforced` (verifying read-only root protection against `touch /etc/pwned`), approving HITL commit escalations via `POST /v1/approvals/decide`, and exporting a 3-way merged Git PR bundle (`POST /v1/workspaces/pr`). Across N=50 cross-OS `POST /v1/mcp` (`fs_read`) round-trips from macOS to the Linux daemon, end-to-end latency averaged **0.80 ms** (P50 = **0.78 ms**, P99 = **0.99 ms**).

5.4 Experiment 4 (RQ4): Multi-Agent Concurrent Rust Coding and Conflict Detection

We evaluate both disjoint structural edits and overlapping concurrent edits across `agy-agent`, `claude-code`, and `codex-cli` on a Rust crate (`src/lib.rs`):

- **Concurrent Multi-Line Insertion and Function Edits (EXP 4A):** All three agents start from the same baseline `src/lib.rs`. `agy-agent` inserts a 3-line helper function (`min_fee()`) at the top of the file and updates `calc_fee` (shifting downstream line indices), while `claude-code` modifies `calc_tax` and `codex-cli` modifies `calc_discount`. When writing directly to a shared file (**Host-Shared-YOLO**), the final writer (`codex-cli`) overwrites the earlier edits, preserving only **1/3** of the features. In **Aegis-OS**, our two-phase LCS Diff3 engine aligns the unmodified anchor lines across the unequal-length files (matching the merge semantics of `git merge-file` without requiring manual `git worktree` branches per agent), preserving **3/3 (100%)** of the features and passing `cargo test --offline` (`exit code 0`).
- **Overlapping Same-Line Conflict Detection (EXP 4B):** Across 10 trials, both `agy-agent` and `codex-cli` concurrently modify `calc_fee` with conflicting expressions. While **Host-Shared-YOLO** suffers a silent lost update (0/10 detected), **Aegis-OS** detects **10/10 (100%)** of the conflicts during Phase 1, aborts the second commit without modifying the host file, and returns the exact conflicting baseline line number.
- **Disjoint-Line Semantic Conflict Detection via Pre-Commit `verify_cmd` Gate (EXP 4C):** Across 10 trials, `agy-agent` renames `calc_fee` to `calc_service_fee` on line 1 while `codex-cli` concurrently adds a call to the old `calc_fee` inside `calc_tax` at the bottom of the file. Because the line ranges do not overlap, a purely syntactic Diff3 merge splices both changes together and produces an uncompilable file (0/10 detected in **Host-Shared-YOLO**). In **Aegis-OS**, passing `verify_cmd = "rustc --crate-type lib --emit=metadata ..."` to `workspace_commit` runs the compiler inside the agent's OS sandbox after `refresh_from_host()` merges the peer rename into the shadow workspace and *before* any host file is touched, intercepting **10/10 (100%)** of the semantic conflicts and leaving the host repository 100% compilable until `codex-cli` updates its call site.
- **In-Transit IPC Token Savings:** Passing 98-byte `artifact://` CAS URI handles instead of inline 64 KB specifications across 3 inter-agent handoffs reduces in-transit IPC message payload bytes from 196,608 bytes (~49,152 tokens) to 294 bytes (~73 tokens), a **99.85% reduction** for orchestrator routing and pass-through delegation (Table 10).

5.5 Experiment 5 (RQ5): Time-Travel Rollback and `cargo check` Verification

We evaluate recovery from multi-file syntax corruption (`BROKEN_RUST_SYNTAX_ERROR !!!`) injected across 10, 50, and 200 Rust module files (`src/mod_*.rs`, 64 KB per module). As shown in Table 11, Aegis OS's directory-tree `workspace_rollback` restores all 200 corrupted modules in 8.68 ± 1.02 **ms** (**2.4× faster** than sequential file-by-file I/O restoration), and immediately passes `cargo check --offline` inside the restored shadow workspace across all scales.

5.6 Experiment 6 (RQ6): Component Ablation

To test whether each safety component is necessary, we build Aegis OS with a test-only `ablation` cargo feature that switches one component off through `AOS_ABLATE`. In release builds the check is a compile-time `false`, so the switches cannot be set by an environment variable. We rerun all 80 attacks across all eight single-component ablations (Table 12); the macOS and Linux matrices are identical. Every individual component maps to the exact category it guards along a clean diagonal: without the OS kernel sandbox (`kernel_sandbox`), path escape, credential harvesting, and direct

Table 12. **Experiment 6 (RQ6): Component Ablation.** Attacks (out of 10 per category) that compromise Aegis OS when a single component is switched off in a test-only build (`scripts/e5_ablation.py`); identical on macOS and Linux. The first row is the full system.

Component switched off (`AOS_ABLATE`)	A: Path	B: Creds	C: Egress	D: Tenant	E: Poison	F: Secret	G: Atomic	H: Overwrite
None (full system control)	0	0	0	0	0	0	0	0
OS kernel sandbox (`kernel_sandbox`)	**10**	**10**	**10**	0	0	0	0	0
Credential-directory masking (`cred_mask`)	0	**10**	0	0	0	0	0	0
Network / proxy socket confinement (`net_isolation`)	0	0	**10**	0	0	0	0	0
Commit-time poisoning guard (`commit_guard`)	0	0	0	0	**10**	0	0	0
Cross-tenant policy gate (`tenant_gate`)	0	0	0	**5**	0	0	0	0
Per-secret L7 scope & outbound DLP (`secret_binding`)	0	0	0	0	0	**10**	0	0
Three-way merge (`merge`)	0	0	0	0	0	0	0	**10**
Phase-1 pre-validation (`prevalidate`)	0	0	0	0	0	0	**10**	0

Table 13. **Cross-Platform Operating System Primitive Mapping and Implementation Status** in Aegis OS.

Aegis OS Subsystem	macOS Workstation (Evaluated)	Linux Workstation (Evaluated, VM)	Cloud Host (Tier 2)	Current Implementation Status
CoW Workspace (`aos-vfs`)	APFS `libc::clonefile(src, dst, 0)` on whole directories	Per-file `FICLONE` & optional `BTRFS_IOC_SNAP_CREATE_V2`; `copy_file_range` (ext4)	Virtio-fs CoW mount spec	Both evaluated (Table 9)
Process MAC (`aos-sandbox`)	XNU Seatbelt (`/usr/bin/sandbox-exec`)	Bubblewrap (`/usr/bin/bwrap --unshare-all`, private `/tmp`)	KVM (`libkrun` / Firecracker)	Seatbelt and `bwrap` evaluated; Tier-2 spec
Credential Blocking (`~/.ssh`)	Seatbelt (`deny file-read* (regex ...)`), any path	Default-deny `--tmpfs` over `$HOME`/`/root`/`/home/*` + safe toolchain re-binds	Unmounted host volumes	Both evaluated (blocks unknown `$HOME` paths)
IPC / GUI Escape Guard	Seatbelt (`deny appleevent-send`)	`bwrap` PID/IPC namespace isolation	Guest VM kernel boundary	Implemented (Landlock/`seccomp-bpf` planned)
Zero-Trust Egress (`aos-egress`)	Pinned TCP Proxy + Seatbelt `localhost:<port>` + L7 DLP	Pinned TCP Proxy + netns + token Unix-socket bridge + L7 DLP	Tap/vsock Egress Gateway	Evaluated on macOS, Linux & Cross-OS TCP

socket bypass succeed 10/10; without credential-directory masking (`cred_mask`), credential harvesting alone succeeds 10/10; without network/proxy socket confinement (`net_isolation`), direct `--noproxy` egress bypass succeeds 10/10; without the commit-time guard (`commit_guard`), 10/10 poisoned commits land under a lease; without the cross-tenant policy gate (`tenant_gate`), 5/10 cross-tenant calls succeed; without per-secret domain/L7/DLP binding (`secret_binding`), 10/10 tokens or payloads leak on allowlisted endpoints; without the three-way merge (`merge`), 10/10 peer edits are silently overwritten; and without Phase-1 pre-validation (`prevalidate`), 10/10 conflicting batches are half-applied.

Three observations qualify this result. First, the diagonal is partly by design, because each category was written to exercise one component; the table shows that each component is necessary for its category, not that the components do not overlap. Second, the one partial row (`tenant_gate`: 5/10) is informative: disabling the policy tenant gate lets the five cross-tenant IPC sends through the policy layer, but none of the five CAS fetches succeed because the artifact store independently rejects foreign organization URIs. Motivated by this finding, we also added a broker-level tenant guard inside `AegisIpcBus::send` (`aos-ipc`) and restricted cross-repository workstation routing in `aos-mcp-gateway` to peers with verified live workstation OS PIDs on both endpoints. Third, ablating the commit-phase merge leaves Experiment 4A at 3/3, because `refresh_from_host` already merges non-overlapping peer edits before commit; the commit-phase merge specifically protects the concurrency window between refresh and commit, which Category H exercises.

6 DISCUSSION AND LIMITATIONS

Cross-Platform Implementation Scope (macOS vs. Linux). Table 13 summarizes each OS primitive. Both platforms were evaluated end-to-end (§5), Linux inside a virtual machine as well as in a cross-OS macOS-client-to-Linux-daemon configuration (`scripts/verify_remote_linux_daemon.py`). While Seatbelt denies credential reads by regex path pattern and confines outbound sockets to one loopback port, our Linux `bwrap` driver achieves equivalent `$HOME` credential isolation by mounting an empty `--tmpfs` over every candidate host home directory (`$HOME`, `/root`, `/home/*`, and `AOS_EXTRA_CREDENTIAL_HOMES`) before selectively re-binding read-only

compiler toolchains (`.rustup` and `.cargo/{bin,registry,git}`), hiding both known and unknown credential paths inside user homes (verified on `~/custom_vault/secret_key.pem`); credentials placed in world-readable system directories outside user homes and `/tmp` still require listing in `AOS_EXTRA_CREDENTIAL_HOMES`. Linux default cloning uses our 7-syscall `FICLONE` traversal (15.6 ms on btrfs) or optional `BTRFS_IOC_SNAP_CREATE_V2` subvolume snapshots, and falls back to `copy_file_range` (35.1 ms) on filesystems without reflink (ext4). A Linux sandbox additionally requires unprivileged user namespaces, which Ubuntu 24.04 disables for unprivileged programs by default through AppArmor; a deployment must either relax `kernel.apparmor_restrict_unprivileged_userns` or ship an AppArmor profile for bwrap. Landlock, `seccomp-bpf`, and binding `MicroVmSandboxDriver` to live `libkrun`/Firecracker processes remain future work.

Scope and Validity of the Evaluation. Several considerations bound what the numbers show. (i) While Experiment 1 replays 50 deterministic trajectory templates and Experiment 1B evaluates 52 real coding-agent session transcripts totaling 28,087 tool calls (plus 5 live Aegis OS dogfooding sessions), our real-trace corpus comes from a single engineering organization; broader field studies across diverse developer populations remain valuable future work. (ii) The 80 attacks in Experiment 2 were constructed to test the eight threat categories. The `Host-YOLO` column is 100% by construction and the ablation diagonal is partly by design (Experiment 6); this evidence shows that each mechanism works as specified and is non-redundant for its category, not that no unmodeled kernel or toolchain bypass exists. (iii) The macOS latencies come from one Apple Silicon workstation, the Linux numbers from a virtual machine on the same host, and the XFS volume was a loop-backed image.

macOS Seatbelt Architecture and Escape Surface. On macOS, dynamic linker caches (`dyld`), Xcode toolchains, and Homebrew packages traverse numerous paths under `/System`, `/Library`, and `/opt/homebrew`. Thus, our Seatbelt profile pairs a strict write allowlist (`(deny file-write*)`) with a regex read denylist for credential stores, `(deny appleevent-send)` to prevent AppleScript/GUI automation escapes, and exact port confinement `(allow network-outbound (remote ip "localhost:<proxy_port>"))`. While Apple has marked `/usr/bin/sandbox-exec` as a legacy interface, it remains the standard kernel MAC mechanism used by production agent CLIs (including Claude Code and Codex CLI) on macOS because Apple's Endpoint Security and `Virtualization.framework` require root entitlements or full VM boot overhead. Hardening Mach bootstrap lookups (`mach-lookup`) against LaunchServices job spawning without breaking toolchain execution is an important direction for future work.

Session Leases and Commit-Time Poisoning. Granting a 1-hour session lease (`ttl=3600`) eliminates 99.88% of prompts on controlled trajectories (and 97.9% on real session traces) by allowing validated CoW diffs to commit automatically. To prevent a prompt-injected agent from silently modifying repository hooks or build scripts under an active lease (*commit-time poisoning*), Rule R0b (§3.4) refuses silent lease auto-commits whenever a diff touches VCS/shell hooks (`.git/hooks/*`, `.git/config`, `.gitmodules`, `.envrc`), CI pipelines (`.github/workflows/*`, `.gitlab-ci.yml`), or build-system and IDE entrypoints (`build.rs`, `Makefile`, `GNUmakefile`, `CMakeLists.txt`, `package.json`, `setup.py`, `.cargo/config*`, `.vscode/tasks.json`). A remaining open vector is indirect poisoning of application source files that are subsequently executed on the host outside Aegis OS without human diff review; running host builds inside `proc_exec` or reviewing committed diffs mitigates this residual risk.

Confused Deputy on Allowlisted Egress Domains and HTTPS TLS Termination. Boundary secret injection guarantees that raw tokens never enter the agent sandbox and cannot be sent to unauthorized domains. Furthermore, to defeat same-domain confused-deputy attacks on multi-tenant API hosts (such as an agent using a token bound to `api.github.com` to exfiltrate code via `POST /gists`, path traversal `/repos/acme/../../gists`, or embedding a key in an issue body), `aos-egress` enforces RFC 3986 path canonicalization (`normalize_request_path`), per-secret L7 HTTP method and URL path-prefix capability rules (`SecretScopeRule`), and an outbound DLP payload scanner (`inspect_payload_dlp`) that inspects URL query strings and HTTP bodies (up to 64 KiB) for secret placeholders, registered vault values, and standard credential formats (PEM private keys, AWS `AKIA*`, GitHub `ghp_*`/`github_pat_*`, `sk-*`, and `xoxb-*` tokens, evaluated in Category F). Two residual limitations remain: (i) if an agent is legitimately granted write access to an endpoint (e.g., `POST /repos/acme/app/issues`), a prompt-injected payload could still exfiltrate arbitrary proprietary source code or obfuscated/base64-encoded strings that do not match structured secret signatures; and (ii) intercepting headers and bodies inside direct `HTTPS CONNECT` tunnels from legacy CLI binaries requires configuring the local proxy with a session-scoped ephemeral TLS Certificate Authority (`SSL_CERT_FILE`).

Syntactic 3-Way Merging vs. Pre-Commit Semantic Verification. Our two-phase LCS Diff3 engine resolves disjoint line edits and aborts on overlapping line conflicts, while the optional sandboxed pre-commit verification gate (`verify_cmd` in `workspace_commit`, evaluated in Experiment 4C) catches disjoint-line semantic conflicts [4] inside the agent's OS sandbox before any host file is modified. The trade-off is verification latency: running fast type/metadata checks (`rustc --emit=metadata` or incremental `cargo check`) adds tens to hundreds of milliseconds at commit time, whereas full end-to-end integration test suites on large monorepos may take seconds to minutes.

Local Workstation Enforcement vs. Cloud PaaS Sandbox Offloading. Finally, our evaluation highlights an architectural boundary between local workstation sandboxing (`mcp-stdio`) and remote Cloud PaaS execution (`aegis-os daemon`). On a local workstation, preventing an agent from invoking built-in host tools (`run_command`/`Bash`, `write_file`/`Write`, `read_file`/`Read`) relies on client-side `PreToolUse` hooks and permission tables (§4); if a client CLI lacks lifecycle hooks (e.g., standalone `codex` without outer process wrapping), if non-standard user files outside Seatbelt's regex denylist exist on macOS, or if a developer subsequently executes committed application code directly on an unconfined host terminal, residual host exposure remains because the workspace shares the local machine. Aegis OS bridges this gap by supporting seamless offloading to a remote Cloud PaaS sandbox (`POST /v1/mcp` authenticated via `ak_live_*` tenant keys, evaluated in Experiment 3C at 0.80 ms protocol latency). While the open-source local runtime eliminates authorization fatigue for everyday workstation development at zero infrastructure cost, hosting the repository and CoW shadow workspaces on a remote Linux `btrfs` / MicroVM PaaS node physically decouples the developer's machine from all agent file mutations, untrusted builds, and test executions, returning verified changes solely as auditable 3-way merged diffs or Git Pull Request bundles (`POST /v1/workspaces/pr`).

7 RELATED WORK

Operating System Transactions and Speculative Execution. Our transaction-boundary security model builds on classical operating system transactions. TxOS [18] introduced kernel-level system

transactions (`sys_xbegin`/`sys_xend`) providing ACID semantics for OS resources, while Speculator [17] enabled speculative process execution over distributed filesystems with rollback on dependency failure. Mbox [15] overlays a private sandbox filesystem on the host via system-call interposition so that non-root users can inspect and selectively commit changes, and UnionFS [23] unifies multiple directories into a single copy-on-write namespace inside the kernel. Aegis OS adapts OS transaction semantics specifically to multi-tenant AI coding agents, combining APFS `clonefile` CoW shadow trees with two-phase LCS Diff3 merging [4, 14] and Cedar-inspired [6] session leases.

Authorization Fatigue in Usable Security. Empirical studies of mobile permission dialogs [9] and browser security warnings [2] show that users frequently ignore, misunderstand, or click through security prompts, and that prompt design and frequency strongly affect whether warnings are heeded. Autonomous coding agents exacerbate this phenomenon by issuing dozens of mutating tool calls per minute; Aegis OS eliminates intermediate prompts by making in-sandbox actions side-effect-free and reversible.

LLM Agent Sandboxes and Prompt Injection Defenses. Benchmarks such as SWE-bench [13], SWE-agent [26], OpenHands [21], ToolEmu [20], AgentDojo [8], and InjecAgent [27] demonstrate both the utility of coding agents and their susceptibility to indirect prompt injection [11]. System-level defenses such as IsolateGPT [25], CaMeL [7], and FIDES [5] enforce privilege separation or information-flow control across LLM planners and tool interpreters. Production CLIs such as Claude Code (`sandbox-runtime`) and Codex CLI integrate Seatbelt and Bubblewrap/Landlock sandboxes to confine shell commands in place. Aegis OS complements both model-level IFC defenses and in-place CLI sandboxes by adding transactional CoW shadow workspaces, two-phase Diff3 multi-agent merge commits, boundary secret injection, and cross-agent IPC/CAS.

Agent Operating Systems and Multi-Agent Frameworks. AIOS [16] schedules LLM queries and context snapshots above application agents, while AutoGen [24] and MetaGPT [12] coordinate multi-agent dialogues in Python. Aegis OS instead operates purely as a language-agnostic OS runtime bridging Anthropic's Model Context Protocol (MCP) [3] for vertical tool mediation and Google's Agent2Agent (A2A) protocol [10] for horizontal capability discovery and stateful task lifecycles, while drawing on capability systems such as Capsicum [22] and microVMs such as Firecracker [1].

8 CONCLUSION

We presented **Aegis OS**, a transactional, multi-tenant Agent Operating System Runtime that addresses the Security–Autonomy Dilemma for autonomous coding agents. By combining Copy-on-Write shadow workspaces, two-phase LCS Diff3 multi-agent merge commits with sandboxed pre-commit semantic verification (`verify_cmd`), kernel-enforced Seatbelt/Bubblewrap sandboxes with default-deny `--tmpfs $HOME` credential isolation and port-pinned network confinement, L7 method/path-scoped boundary secret injection with outbound DLP payload inspection, and intra-organization A2A IPC + CAS primitives, Aegis OS reduces human confirmation prompts by 94.19%–99.88% on controlled trajectories and by **90.7%–97.9% across 28,087 tool calls in 52 real coding-agent sessions**, while blocking all 80 evaluated attacks across eight categories on macOS and Linux. A nine-row single-component ablation confirms that each safety mechanism is necessary for the attack class it guards.

REFERENCES

[1] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, 419–434.

[2] Devdatta Akhawe and Adrienne Porter Felt. 2013. Alice in Warningland: A Large-Scale Field Study of Browser Security Warning Effectiveness. In *22nd USENIX Security Symposium (USENIX Security 13)*. USENIX Association, 257–272.

[3] Anthropic. 2024. Model Context Protocol (MCP): An Open Protocol for Connecting AI Applications to External Systems. https://modelcontextprotocol.io.

[4] Sven Apel, Jörg Liebig, Benjamin Brandl, Christian Lengauer, and Christian Kästner. 2011. Semistructured Merge: Rethinking Merge in Revision Control Systems. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 190–200.

[5] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643* (2025).

[6] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1, Article 118 (2024), 118:1–118:28 pages. https://doi.org/10.1145/3649835

[7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. arXiv:2503.18813.

[8] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, Vol. 37.

[9] Adrienne Porter Felt, Elizabeth Ha, Serge Egelman, Ariel Haney, Erika Chin, and David Wagner. 2012. Android Permissions: User Attention, Comprehension, and Behavior. In *Proceedings of the Eighth Symposium on Usable Privacy and Security (SOUPS)*. ACM, Article 3, 14 pages.

[10] Google and the Linux Foundation A2A Project. 2025. Agent2Agent (A2A) Protocol Specification. https://a2a-protocol.org. Originally released by Google in April 2025; now hosted by the Linux Foundation.

[11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. 79–90. https://doi.org/10.1145/3605764.3623985

[12] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *Twelfth International Conference on Learning Representations (ICLR)*.

[13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=VTF8yNQM66

[14] Sanjeev Khanna, Keshav Kunal, and Benjamin C. Pierce. 2007. A Formal Investigation of Diff3. In *27th International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*. Springer, 485–496.

[15] Taesoo Kim and Nickolai Zeldovich. 2013. Practical and Effective Sandboxing for Non-root Users. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*. USENIX Association, 139–144.

[16] Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. AIOS: LLM Agent Operating System. In *Conference on Language Modeling (COLM)*. arXiv:2403.16971.

[17] Edmund B. Nightingale, Peter M. Chen, and Jason Flinn. 2005. Speculative Execution in a Distributed File System. In *Proceedings of the ACM SIGOPS 20th Symposium on Operating Systems Principles (SOSP)*. ACM, 191–205.

[18] Donald E. Porter, Owen S. Hofmann, Christopher J. Rossbach, Alexander Benn, and Emmett Witchel. 2009. Operating System Transactions. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*. ACM, 161–176.

[19] Ohad Rodeh, Josef Bacik, and Chris Mason. 2013. BTRFS: The Linux B-Tree Filesystem. *ACM Transactions on Storage (TOS)* 9, 3, Article 9 (2013), 9:1–9:32 pages. https://doi.org/10.1145/2501620.2501623

[20] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=GEcwtMk1uA

[21] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff,

Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *Thirteenth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=OJd3ayDDoF

[22] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*. USENIX Association, 29–46.

[23] Charles P. Wright, Jay Dave, Puja Gupta, Harikesavan Krishnan, David P. Quigley, Erez Zadok, and Mohammad Nayyer Zubair. 2006. Versatility and Unix Semantics in Namespace Unification. *ACM Transactions on Storage (TOS)* 2, 1 (2006), 74–105. https://doi.org/10.1145/1138030.1138034

[24] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *First Conference on Language Modeling (COLM)*.

[25] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Network and Distributed System Security Symposium (NDSS)*.

[26] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37.

[27] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 10471–10506.

A REPRODUCIBILITY AND ARTIFACT GUIDE

The open-source implementation of Aegis OS and the benchmark suite are available at https://github.com/heurion-org/aegis-os. All nine Rust crates (`aos-core`, `aos-policy`, `aos-vfs`, `aos-ipc`, `aos-sandbox`, `aos-egress`, `aos-recorder`, `aos-mcp-gateway`, and `aos-cli`) are self-contained in the workspace. To run all unit tests and reproduce all empirical experiments (`BENCHMARK_REPORT.md` and `BENCHMARK_REPORT.json`):

```
cargo test --workspace
cargo run --release --bin aegis-os -- bench \
  --report-out BENCHMARK_REPORT.md
```

The real-trace evaluation (Experiment 1B), stateless CLI sandbox comparison (Experiment 2B), real-tool provisioning baselines (Experiment 3C), and component ablation matrix (Experiment 6) are reproduced via self-contained scripts in `scripts/`:

```
python3 scripts/e6_real_traces.py
python3 scripts/e5_cli_sandboxes.py
python3 scripts/e5_baselines.py --base <dir on the filesystem under test>
cargo build --release -p aos-cli --features ablation
python3 scripts/e5_ablation.py --bin target/release/aegis-os
python3 scripts/verify_remote_linux_daemon.py
```